

MATLAB PF WRAPPER

Documentation for using PF32 camera with Matlab



SYSTEM REQUIREMENTS

This wrapper has been designed for Mathworks Matlab 2017b running on a Windows 10. There is no guarantee of full functionality with older versions of Matlab or Windows. The wrapper requires a powered up PF32 camera connected to a USB 3.0 port of the operating computer.

INSTALLATION

The wrapper is included in the PF32 application installer. Once installed, the Matlab files are placed by default in the Program Files folder. In most cases this would be:

```
C:\Program Files\Photon Force\Matlab\wrapper
```

The Photon Force installation folder contains two relevant subfolders:

Matlab\wrapper	Contains Matlab scripts
PF32_API	Contains Windows libraries

It is advisable to add a line into the `startup.m` file to enable Matlab use the wrapper files regardless of the active path. Create or amend existing the `startup.m` file by typing the following line in the Matlab Command Window:

```
edit([userpath '\startup.m'])
```

This will open the `startup.m` file for editing. Add the following line to the `startup.m` file:

```
addpath([getenv('programfiles'), '\Photon Force\Matlab\wrapper\']);
```

Save the `startup.m` and run it or restart Matlab.

PF WRAPPER OPERATION

PF32 wrapper is a set of Matlab m-files that operate the PF32 camera. Number of the m-files are intended for end-user application. The best way to get started is to run the function:

`pf_example`

`pf_example` is the function that is the ideal starting point to the use of the PF wrapper. The function runs a series of examples:

1. Connect the camera using the `pf_open` function.
2. Report the status of the library and the connected device using the `pf_getLibraryStatus` function. It is a debugging tool, which prints a report in the Command Window. If problem or crash are encountered, the last printed line message should point to what is broken.
3. Acquire a series of 100 intensity (photon count) images and display them on the screen.
 - a. Set number of frame accumulations to 20 by calling the `pf_setNumberAccumulations` function.
 - b. Set the frame integration time to 50 microseconds by calling the `pf_setExposure` function.
 - c. Stream a hundred frames on the computer screen using the `pf_liveImageStream` function.
4. Change the PF32 camera to the TCSPC mode. The camera assumes a laser SYNC input connected to drive the TDC stop signal.
 - a. The camera mode is set by the `pf_setMode(pf32, 'TCSPC_laser_master')` call.
 - b. If no laser is available, the above line of code can be changed to use the internal clock by calling `pf_setMode(pf32, 'TCSPC_sys_master')` instead.
5. Acquire a series of 10 000 TCSPC frames.
 - a. Set number of frame accumulations to 1 by calling the `pf_setNumberAccumulations` function.
 - b. Set the frame integration time to 2 microseconds by calling the `pf_setExposure` function.
 - c. Measure the laser clock frequency and duty cycle on the PF32 camera input by calling the `pf_getSyncFrequency` function.
 - d. Rapidly acquire 10000 frames the `pf_getMultipleFrames` function.
6. Display the TCSPC data as an image of the scene and a pixel-by-pixel histogram using the `pf_displayHistogram` function.
7. Disconnect the camera and the library using the `pf_close` function.

The functions covered in the `pf_example` are recommended for use by the end-users. Other functions included in the PF wrapper are for internal functionality and are not recommended for use. Their misuse may lead to malfunction.

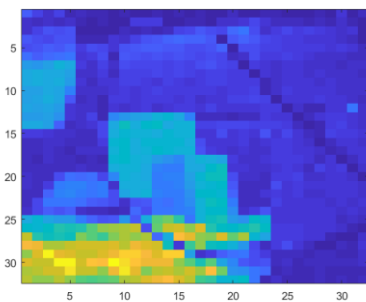


Fig. 1. Photon Counting mode shows the scene image.

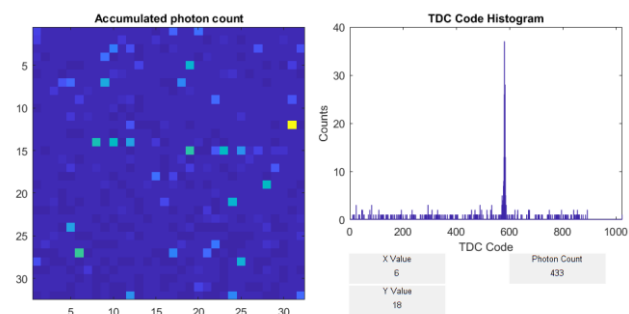


Fig. 2. TCSPC mode shows the scene image on the left and the TDC code histogram on the right. In the above example, the illumination was achieved with a pulsed diode laser with a 20 MHz repetition rate.